Implementing a Game with Containers

How §3–§5 of *Containers for Typed Agentic AI* became `stalin`

Robin Langer

1 September 2026

1 The claim

A turn-based game is a container, and not by analogy.

$$P = \text{the situations that can arise} \qquad R\,s = \text{the moves that are legal in } s$$

That is Ghani's own third example, verbatim: "a data structure and its valid pointers — the positions you are allowed to navigate to." And a *player* — human or machine — is exactly a program of type $(s : P) \to R\,s$, which is §3's "the programmer's job". A game engine that validates moves at run time is a game engine whose container it has not written down.

So the thing a game can demonstrate that a bookkeeping assistant cannot is sharper than "the types fit together":

> An illegal move should be *unconstructible*, not rejected.

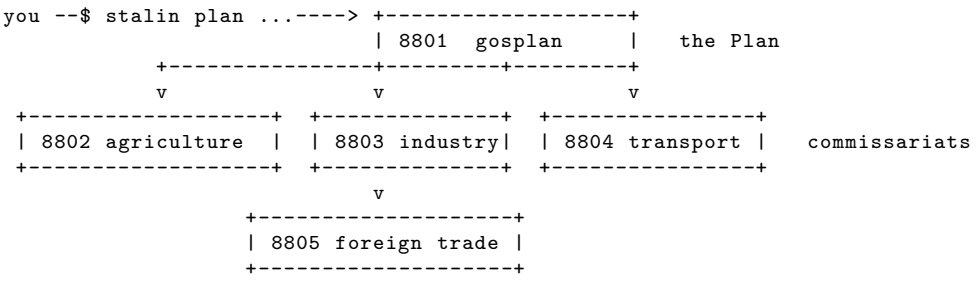
`stalin` is a resource-allocation game about the First Five-Year Plan, built to see how much of that is actually attainable in TypeScript. This note records what worked, what had to be discovered, and — §7 — what did not.

2 The tower is a command economy

§4 says an event handler is a morphism of containers: a pair

$$u : P \to Q, \qquad f : (p : P) \to T\,(u\,p) \to R\,p,$$

control flowing down through u, data flowing back up through f. Relabel: Gosplan issues a quota to the Commissariat of Heavy Industry, which issues one to the mill, which reports what it made. That is not a metaphor for a command economy; it is one.



Each arrow is a spawned CLI process, not a function call, which matters for §7: between any two boxes everything the type system knew is destroyed, and each ingress has to re-establish it by validating rather than asserting.

3 The containers, written as their graphs

A container is presented not as a pair (`Shape`, `Pos`) but as its **graph**: the union of its fibres. That is what makes the type-level constructions compute, because a conditional type distributes over a union.

```
interface Fib<S, P> { readonly shape: S; readonly pos: P }
type Shape<C> = C extends Fib<infer S, unknown> ? S : never;
type PosAt<C, S> = C extends Fib<infer S1, infer P> ? (S extends S1 ? P : never) :
    never;
```

Each commissariat is then a declaration you can read against the server it describes — an indexed family, one line per index:

```
type AgricultureC =
  | Fib<{tag:"Harvest"; workforce: Workforce; tractors: number; year: number},
      HarvestReport>
  | Fib<{tag:"Winter"; ruralRations: Grain; industrialRations: Grain; year: number},
      WinterReport>
  | Fib<{tag:"Report"; year: number}, Dispatch>
  | Fib<{tag:"Census"}, CensusReturn>;
```

Two things are worth noticing. First, nothing says a server must correspond to exactly one container. Port 8803 serves three: the smelter, and separately

```
type TractorWorksC = Fib<{tag:"BuildTractors"; steel: Steel; plantCapacity: number;
     year: number}, TractorReport>;
type ArmamentsC = Fib<{tag:"BuildArmaments"; steel: Steel; year: number},
    ArmamentReport>;
```

They are split precisely so the Plan can take their *product* (§4.3). A sub-container is just a sub-family; the split costs nothing at run time and buys a combinator.

Second, `Report` and `Census` have different response types but the same standing — and that is where §6 comes back. See §6 below.

4 The four combinators

4.1 Tensor ⊗ — the three sectors

The fields, the mill and the railway all work through the same year, and all three owe a report. There is no partial year.

```
const fieldsAndMill = tensorC(agri, smelter);
const produce = tensorC(fieldsAndMill, transport);
```

Shapes multiply and *positions multiply*: a shape of `produce` carries a prompt for each sector, and its position carries a report from each. The `Promise.all` inside `tensorC` is not an optimisation, it is the definition.

4.2 Sequential composition ◁ — the haul

What the railway is asked to move cannot be known until the fields have been reaped, because it depends on how much there is and on what grade it came in at. That is exactly ◁: the composite's shape is a first prompt *together with a function* choosing the second from the first's reply.

```
const supply = seqC(produce, transport);

const supplied = await supply.run(
  supply.step(produceShape, (produced) => {
    const h = produced.left.left; // a HarvestReport, at this fibre
    const stateGrain = h.grain * take + s.grain;
    exp = resolveExport(h.grade, order.exportGrain); // <- the gate, SS5
    const offerPort = exp.choice === "none" ? 0 : /* ... */ ;
    return { tag: "Haul", railCapacity: s.railCapacity + produced.right.added,
             needIndustry: indMouths, offerPort, year };
  }),
  depth);

const harvest = supplied.first.left.left; // typed, uncast
const haul = supplied.second;
```

`produced.left.left` is a `HarvestReport` because that is the fibre we prompted, not because anybody asserted it. The application of `next` inside `seqC`'s runner is the whole content of $\triangleleft$: the composite shape carried a function, and that is where it is used.

4.3 $\triangleleft$ again — and this time the player is inside it

The year began as one turn. You allocated labour, set the quota, and disposed of the harvest all at once, which meant the quota was chosen with the harvest already on the table. That is a strange thing to have modelled: a procurement quota that knows the harvest is not a quota, it is a share.

Splitting the year into **spring** and **autumn** fixes it, and the shape of the fix is $\triangleleft$ with the human in the middle:

spring	where the hands go; how hard the villages are squeezed
autumn	export, trade, purchase, and tractors-or-armaments

The spring prompt is answered without knowing the weather. The autumn prompt *is not a fixed prompt at all*: which options exist, and which of them the compiler will accept, is a function of the spring reply — because `ExportChoice<G>` is indexed by the harvest grade, and the grade is not known until the sowing has been run. The composite shape is $(s : \text{Spring} \ast\ast \text{Spring.Pos}\, s \to \text{Autumn})$, which is $\triangleleft$'s shape exactly. Previously that dependency was internal, tucked inside `runYear`; now the player stands at the join and experiences it as the thing it is. The state machine that makes this legible to the interface is one field:

```
type Season = "spring" | "autumn";
type GosPrompt =
  | Fib<{ tag: "Sow"; order: SowOrder }, YearReport> // legal only in spring
  | Fib<{ tag: "Reap"; order: ReapOrder }, YearReport> // legal only in autumn
  | /* Status | Reckoning | Reset */ ...;
```

Ten turns rather than five, and the two halves ask different questions. A quota fixed in March against a harvest that fails in August is not a game mechanic. It is how a famine is administered.

4.4 Product $\times$ — tractors or armaments

The pattern §5 says the practitioners' catalogue had not named: prompt both sides, and owe a reply from exactly one. In the game it is the central strategic decision, because armaments are the only investment with no economic return whatever — they are worth something only if 1941 happens.

```
const build = productC(tractorWorks, armaments,
  // The policy reads the decision off the shape: whichever side was offered
  // the steel is the side that answers.
  ({ a }) => (a.steel > 0 ? "a" : "b"));

const made = await build.run(
  build.offer(
    { tag: "BuildTractors", steel: toTractors ? s.steel : 0, plantCapacity, year },
    { tag: "BuildArmaments", steel: toTractors ? 0 : s.steel, year }),
  depth);

if (made.side === "a") { s.tractors += made.value.built; }
else { s.warReserve += made.value.reserved; }
```

Narrowing on `made.side` is the elimination of `Either` $(R\,p)\,(T\,q)$. This is why the year's steel allocation was made a *decision* rather than a fraction: a split would have been the tensor, and calling it a product would have been false.

4.5 $\times \to \otimes$ — the railway to the port, and why it was removed

This was the most satisfying place the algebra earned its keep, and it is gone. While the line to the port was narrow there was one trade delegation and one contract, so the sale was a *product* — grain or steel. Build enough rail and both could be shipped, so the sale became a *tensor*:

```
const narrowSale = productC(grainSale, steelSale, /* ... */); // removed
const wideSale = tensorC(grainSale, steelSale); // removed
const wide = throughputOf(s.railCapacity) === "wide";
```

The original design goal was "transition from exporting grain to exporting steel, or export both", and in the algebra that last clause was not a bigger number but a different combinator — same shapes, different positions: answer one, or answer both.

The design changed. Selling steel was removed from the game, because a country that exports its steel is not industrialising, and everything the plan is for is made of it. That left both combinators running on a quantity that is permanently zero. They were deleted rather than left in place, since a demonstration that cannot fire demonstrates nothing, and a paper claiming live machinery that no longer runs is worse than a shorter paper. Both combinators are still genuinely exercised elsewhere: $\otimes$ by the three sectors working the same year, $\times$ by tractors against armaments.

What survives is the typed gate. `SteelTrade<P>` is now

```
export type SteelTrade<P extends SteelPosition> =
  P extends "deficit" ? "none" | "buy"
  : "none";
```

which is a fibre with a single inhabitant everywhere but deficit — not a disabled button, but a position type with nothing in it to choose. Removing the capability was itself checked by the compiler: deleting the `SellSteel` decoder while the fibre still existed in `ForeignTradeC` is an error, because the fibre and its handler are one fact stated twice.

4.6 Sum $+$ — present, but not as a call

`sumC` is never invoked in the game, and the note should say so plainly. Routing appears instead as the *shape type* of each container: a tagged union $\{\texttt{Harvest}\} + \{\texttt{Winter}\} + \ldots$ is a coproduct, and its eliminator is the `switch` in each server's dispatch. So the sum is doing real work, in the

same place §3's container always put it — but no composite container is built with it, and it would be an overstatement to claim otherwise.

5 The dependent gate

This is where the game does something the bookkeeping assistant could not.

The algebra gives Σ-types over *finite* index sets, because $\Sigma\,(s : S).\,B\,s = \bigcup_s \{s\} \times B\,s$ and a conditional type distributes over a union. So a rule expressible over a *grade* can be carried by the checker, and the same rule over a tonnage cannot. The game is therefore built to think in grades:

```
type ExportChoice<G extends HarvestGrade> =
  G extends "failure" ? "none"
: G extends "poor" ? "none" | "surplus"
: G extends "adequate" ? "none" | "surplus" | "full"
: "none" | "surplus" | "full" | "maximum";

function exportPlan<G extends HarvestGrade>(grade: G, choice: ExportChoice<G>): ...
```

`G` is inferred from the first argument, so `choice` is typed at *that* grade alone. The consequence:

```
exportPlan("bumper", "maximum"); // fine
exportPlan("failure", "maximum"); // does not compile
```

Exporting grain during a failed harvest is not a move the engine rejects. It is a move that cannot be written down — which was the whole ambition in §1.

The same trick carries the game's arc:

```
type SteelTrade<P extends SteelPosition> =
  P extends "deficit" ? "none" | "buy"
: P extends "balanced" ? "none"
: "none" | "sell";
```

You begin in deficit and buy steel with grain at a punitive rate; you end in surplus and sell it. The transition from importing steel to exporting it is a **change of fibre**, not an `if`, and `balanced` is the hinge at which neither trade exists.

Introduction and elimination. The player types a string, so at the CLI boundary the choice is a `string` and must be checked — there is no avoiding that, and it is the same ingress discipline every hop needs anyway. What the gate buys is everything *past* that boundary. The check is a `switch` on the grade, and each branch is inside one fibre:

```
switch (grade) {
  case "failure": { const c = exportPlan("failure", "none"); /* ... */ }
  case "poor": { const c = exportPlan("poor", ok ? "surplus" : "none"); /* ... */ }
  // ...
}
```

That `switch` is the elimination rule of the finite Σ. Inside a branch the compiler knows which fibre it is in, and no illegal combination can be constructed from there on.

It is also historically apt. Quotas were set against reported categories, never measured tonnes. Making the type system coarse in the same way the bureaucracy was coarse is a coincidence, but a useful one: it means the discretisation the technique demands is not a distortion of the subject.

6 Shape and truth

§6 makes a caveat about the language model at the leaf: it is an untyped oracle, and presenting it as $(s : S) \to P\,s$ is a *promise* made good by a wrapper, not a fact about the model. The game relocates that caveat and makes it a mechanic.

Every commissariat holds two things the Plan cannot see: how well it is actually doing, and how far it will inflate a shortfall.

```
record(year: number, quota: number, delivered: number): Dispatch {
  const gap = Math.max(0, quota - delivered);
  const reported = delivered + gap * this.bias * (0.7 + 0.6 * this.noise());
  // ...
}
```

`Report` and `Census` return values of the same standing: both are well-typed positions of their container, both validated at ingress, both indistinguishable to the checker. One is true. The player plans against the other, and the divergence is revealed only at the reckoning.

> The type of a dispatch guarantees its shape and says exactly nothing about its truth.

That is not a limitation of this encoding; it is what a type system is. A command economy is a typed tower whose leaves are untrusted oracles, and the types cannot save you — which is a serviceable summary both of §6 and of the subject matter.

7 Where it leaks

The $\lhd$ implemented is a sub-container of the paper's. Its fibres are indexed by *pairs* (S_A, S_B): a first shape together with a function landing in one particular second shape. Ghani's $\lhd$ is larger, because there `next` may return different shapes of d for different positions, making the composite position a genuine $\Sigma\,(p : \texttt{c.Pos}\,s).\,\texttt{d.Pos}\,(v\,p)$ — a Σ over an *infinite* index, which does not distribute. Restricting `next` to a single fibre is exactly what buys the position type back as a plain product. What it costs is a branching `next`, which has a union return type and lies in no single fibre.

`next` must be pure, and the game wanted it not to be. The export decision is made inside `next` (it depends on the harvest grade) but is needed afterwards. Since `next` is a function of the position and returns only a shape, the decision is captured in a closure variable and read back after the run. That works and is flagged in the source, but it is a concession: the honest version would carry the decision in the composite shape.

The morphism is still not first class. The container is a value; the event handler over it is not. f must know which branch it is in to know what it owes, so the year is written one phase at a time. Each phase is monomorphic and therefore cast-free, but a genuinely first-class $\mathrm{Cont}((P, R), (Q, T))$ would need the fibre treatment applied to morphisms as well.

A capped substitution is a step function, and a step function is a game that cannot tell plans apart. The war originally read $\min(W,\, m \cdot \texttt{maxSubstitution})$ for materiel and $\min(\texttt{steelPerSoldier}, 1.5)$ for protection. Two ceilings, each individually defensible — a soldier can only carry so much — and stacked they meant that past 44 armaments *every* plan fought an identical war: 22 men called up, 5 of them dead. Four quite different playthroughs scored the same, and the search that was meant to find the best plan reported a four-way tie. Replacing both caps with a complementarity,

$$P(m, W) \;=\; \texttt{manPower} \cdot m \;+\; \texttt{steelPower} \cdot \sqrt{W \cdot m},$$

removes the flat without removing the diminishing returns: the smallest winning m is the positive root of a quadratic in $\sqrt{m}$, falls smoothly with W, and never bottoms out. The old model said a rifle is worth nothing to a man already carrying one; the new one says it is worth less, which is both truer and playable.

Splitting the year silently deleted a strategy. When a year was one turn, labour and the procurement quota went out in the same order, so a plan could staff the mill *and* squeeze the villages. Splitting the year put both in spring and the menu offered them as separate letters — so choosing one repealed the other, and the grain taken by a total quota had no mill to be smelted by. Stripping the villages stopped being a hard choice and became simply bad: 21.8 tonnes of armaments and a lost war. Nothing failed to compile, no invariant was violated, and the game quietly lost the decision it exists to pose. The fix was to make the quota *standing* — a decree in force until repealed, which is also what it was — after which the same intent produced 93.6 tonnes and a war won for four soldiers and forty-three villagers. The lesson is not about types. Mechanical refactors preserve behaviour; they do not preserve *strategy*, and only playing finds out.

`deno check` passed while the game was broken. When the library was vendored into this repository, all files typechecked and the first playthrough died at once, reporting that the delegate to port 8804 had exited with status 1. The reason is that `wire.ts` locates the CLI tool it spawns with

```
new URL("./delegate.ts", import.meta.url) // resolved at run time, never checked
```

and that file had not been copied. The one file that makes each hop an actual process was the one file the compiler could not tell me was missing. Types hold within a process; the moment a boundary is a filesystem path or a socket, only running it tells you the truth.

8 Evidence

```
$ deno check *.ts lib/*.ts      # 0 errors, including type-tests.ts
$ ./playthrough.sh ...          # deterministic on the seed: a game is a regression test
```

`type-tests.ts` asserts the typed rules against the compiler. Every `@ts-expect-error` in it claims that the next line *is* an error, and TypeScript reports an unused directive as an error in its own right, so the file fails both if a legal move is rejected and if an illegal one is accepted. A negative suite that cannot fail is worth nothing, so it was mutation-tested: widening the `"failure"` fibre by one option makes it stop compiling.

That the types are right says nothing about whether the game is any good, so the design's central claim was checked by playing it. Three strategies, same seed:

strategy	letters	starved	fallen	total dead	war
gentle arms *(found by search)*	`AJBEBJAJAJ`	5	4	**9**	won
brutalist mechanisation	`CMKEBIBJNJ`	15	3	**18**	won
no railway, so the grain rots	`KABEBIBJAJ`	25	4	**29**	won
partial armaments	`CABEBINJAA`	21	14	**35**	won
totally brutal	`KJBEAJBJAJ`	44	5	**49**	won
tractors bought abroad	`AMAMAFAMAF`	3	57	**60**	lost
never armed	`CAKEBINIAI`	15	52	**67**	lost

Five of those win, and the two orderings disagree. Ranked by *soldiers* lost, brutalist mechanisation is the best plan on the board — 3 fallen, because it ends with 98 tonnes of armaments, more than any other line, and an army small enough to be lavishly equipped. Ranked by *everyone* lost it comes second, because the fifteen it starved to build that machine outnumber the one soldier it saved. The game holds both numbers and declines to say which is the real one.

The last row of that table is the one the design asks for and does not get. The brief is that *brutalist mechanisation should be optimal*: starve the villages early, spend it on rail, engineers and tractors, then turn the surplus into an army. It builds the best war machine in the game and still loses on total dead to a line that simply smelts. The traces say why:

	mill workforce, 1928–32	armaments
brutalist mechanisation	8, 8, 16, 23, **45**	98, first built in 1932
gentle arms	8, 15, 22, 21, 20	89, first built in 1929

The mechanised economy matures in 1932, and then the plan ends. Its chain is rail → export → engineers → works → tractors → freed hands → mill → steel → armaments: nine links, each costing at least one turn, against ten turns in the plan. The gentle line skips six of them and spends four years arming instead of one. Mechanisation builds the better machine and is given a single year to run it.

No constant fixes this. The search covered enemy strength, subsistence, the grain price, track laid per worker, both engineer constants, all three tractor costs, the works, both yields and the procurement fractions — two hundred evaluations and a coordinate descent. Raising `engineerCapacity` until mechanisation out-produced the gentle line (92 armaments against 89) still lost 18 to 9, because ten extra starved buy back at most one soldier. What is wrong is not a number but a *dependency length*, and the fix is structural: the autumn menu couples `buy` and `build` into one letter although `ReapOrder` carries them as independent fields, so uncoupling them roughly halves the chain in turns. That is the same conflation that had already been fixed in spring, where labour and the quota were competing for one decision.

An earlier balance had brutality *losing* the war, and it took a remark from Robin to see what that was: cruelty punished by defeat is the comfortable version and not what happened. The thesis was not in the design, it was in a `Math.min` capping how far materiel may stand in for men, and in the constant giving the worth of an unarmed soldier. Two numbers were carrying an argument the simulation was then read back as having discovered.

One correction made while writing this document. Two of the four combinators, `supply` and `build`, were originally *declared and never run*: the haul and the steel allocation were direct calls that bypassed them. A document describing them as load-bearing would have been false, so the code was changed rather than the prose. It is worth recording because it is the characteristic failure mode of this kind of work: an algebra can be written down, typecheck, read beautifully, and not actually be in the path of anything.